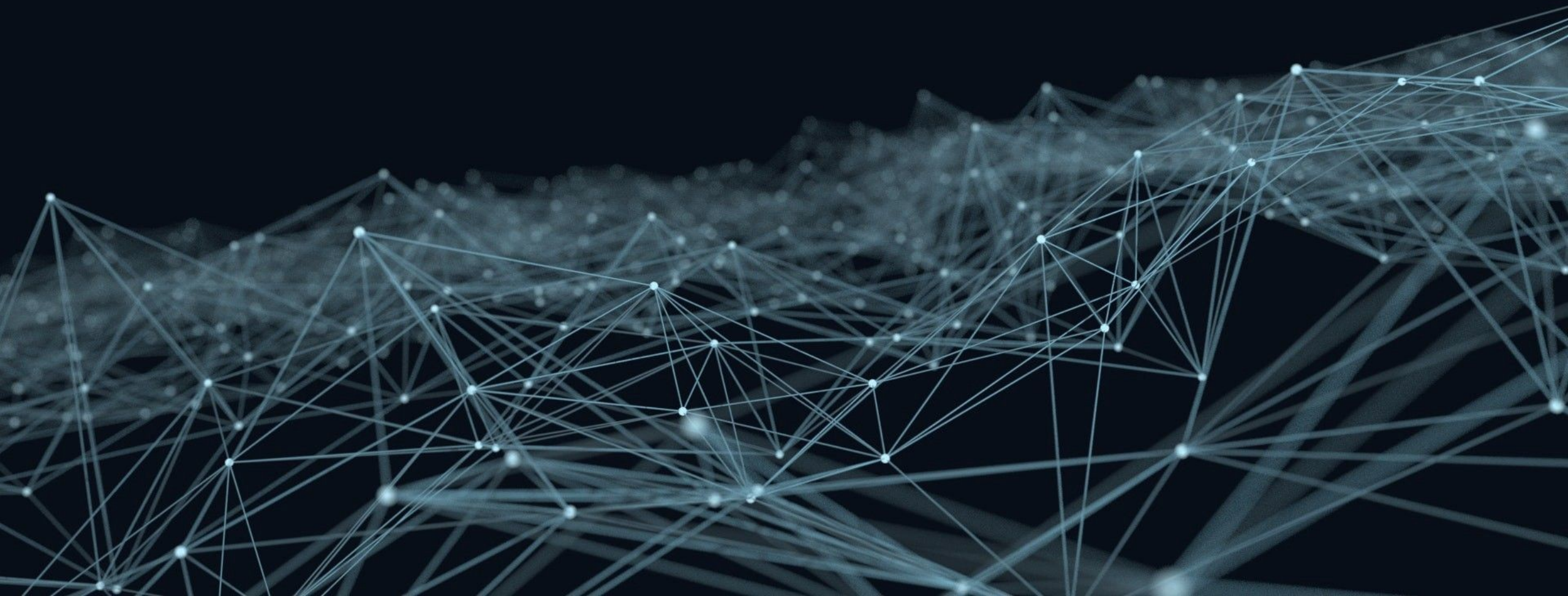
Java Full Stack Development

1. Secure External API Consumption



TEKsystems Global Services

WORKFORCE DEVELOPMENT

Module Topics

- HTTP Clients
- RestTemplate
- WebClient
- Bearer Tokens and Authentication
- Resilience Patterns
- Handling 401/403

HTTP Client Landscape

Modern Service Architecture

- Every production application consumes external services
  - Payment processors, identity providers, data feeds, partner APIs
  - Internal microservices exposed over HTTP
  - Third-party SaaS platforms and cloud provider APIs
  - Legacy systems surfaced behind REST facades

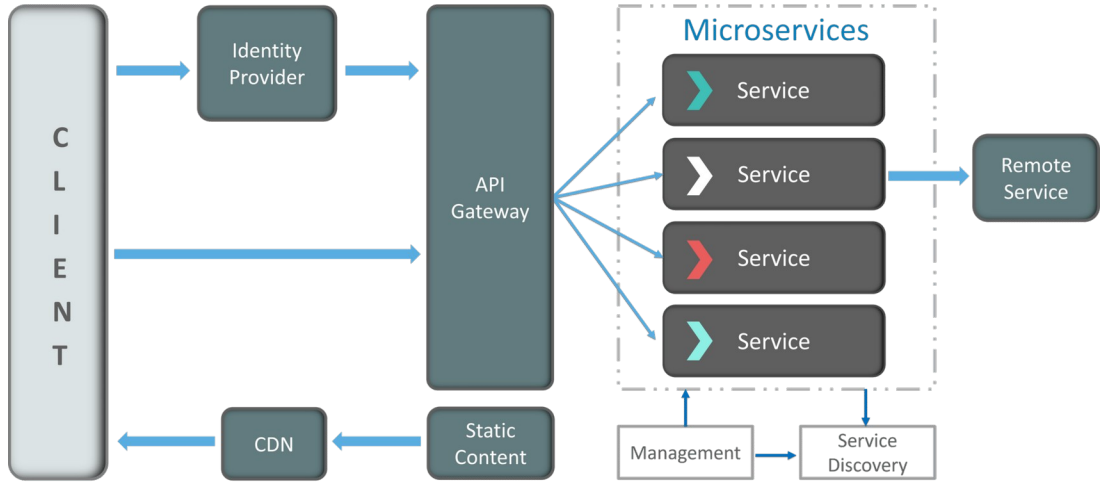


Image Credit: https://peerdh.com/blogs/programming-insights/microservices-architecture-for-modern-web-applications

Modern Service Architecture

- The integration layer is a major attack surface
  - Stolen or leaked tokens in logs and stack traces
  - Credentials hardcoded in source code or config files
  - No retry logic
    - Silent data loss under transient failures
  - No timeout limits
    - Thread exhaustion, cascading failures
  - Blind trust of upstream responses
    - Injection and deserialization risk

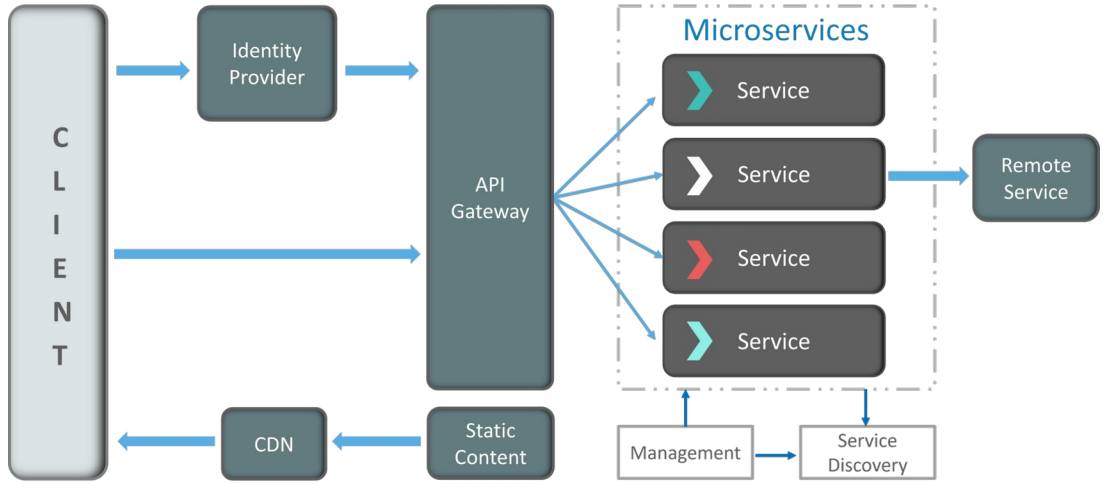


Image Credit: https://peerdh.com/blogs/programming-insights/microservices-architecture-for-modern-web-applications

Modern Service Architecture

- HTTP client code is a security boundary
  - Poorly written client code has been the source of major production incidents and security breaches
  - For example, tokens appearing in application logs, no timeout causing thread pool exhaustion under load

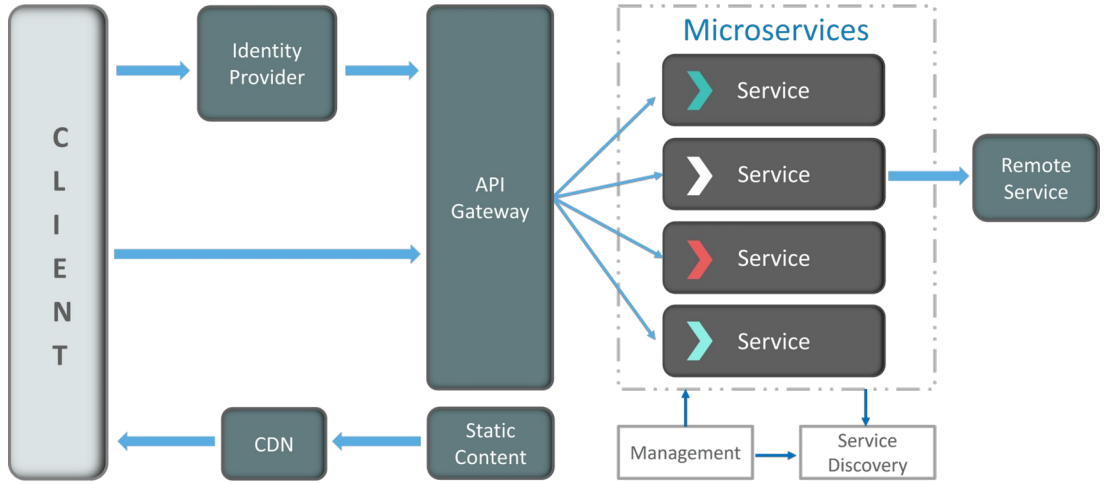


Image Credit: https://peerdh.com/blogs/programming-insights/microservices-architecture-for-modern-web-applications

RestTemplate

RestTemplate

- RestTemplate
  - The original Spring HTTP client
  - Introduced in Spring 3 (2009)
  - Synchronous, blocking: one thread per request
  - Familiar, simple, widely documented
  - Built around a template method pattern with callback hooks
- Still in code bases everywhere
  - Long history means massive community knowledge
  - Many internal enterprise APIs were built against it
  - Simple use cases require very little configuration

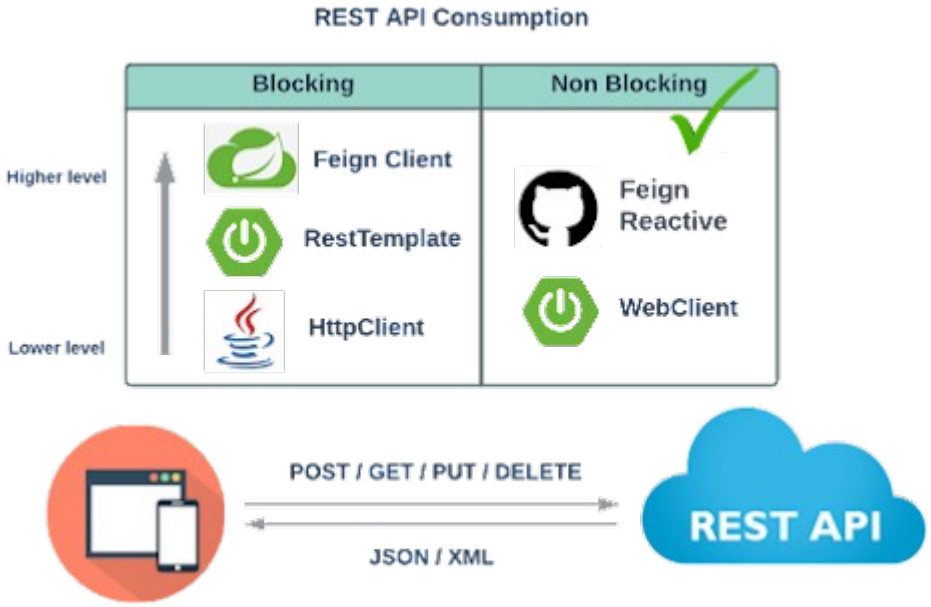


Image Credit: https://javarevisited.blogspot.com/2023/08/difference-between-resttemplate-and.html

RestTemplate

- RestTemplate
  - In maintenance mode as of Spring 5
  - No new features are being added
  - Spring's own documentation recommends migrating to WebClient
  - Will not receive support for newer HTTP features (HTTP/2, streaming, etc

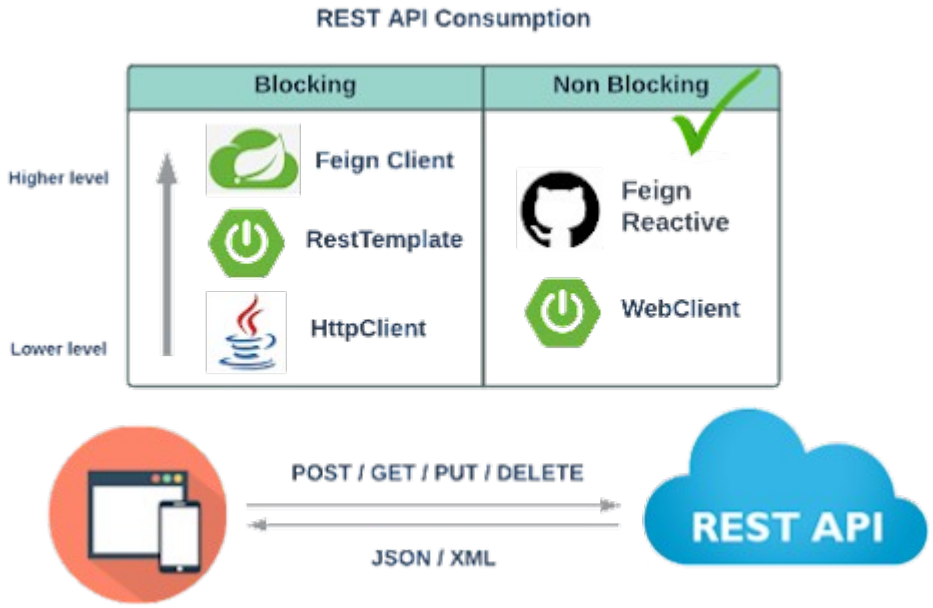


Image Credit: https://javarevisited.blogspot.com/2023/08/difference-between-resttemplate-and.html

RestTemplate

- RestTemplate
  - Predates modern reactive programming
  - Provides a high-level abstraction over raw Java HTTP connections
  - Follows the Spring "template" pattern
    - The same design used in JdbcTemplate, JmsTemplate
  - Handles the repetitive boilerplate of HTTP communication automatically

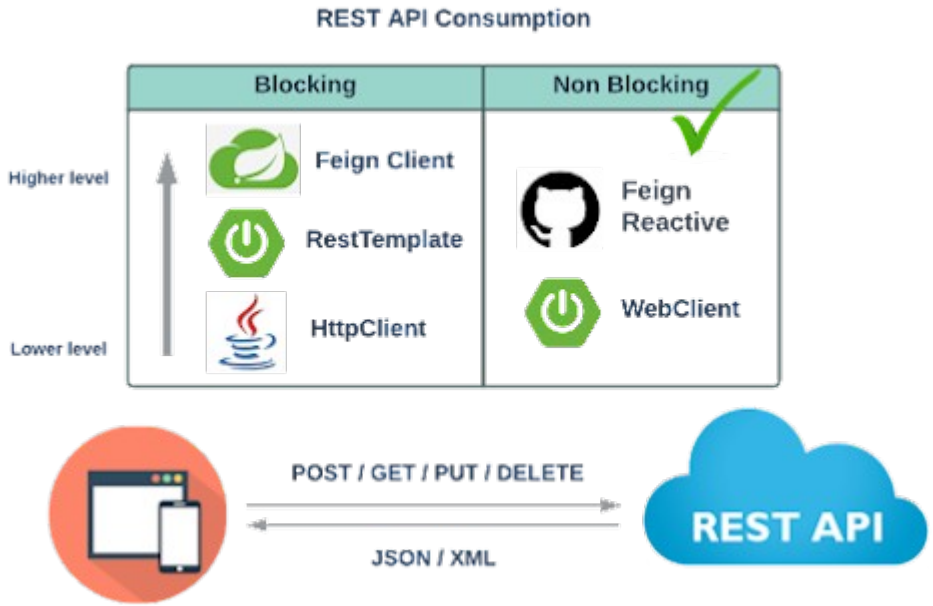


Image Credit: https://javarevisited.blogspot.com/2023/08/difference-between-resttemplate-and.html

RestTemplate

- RestTemplate
  - Abstracts away
    - Opening and closing HTTP connections
    - Serializing Java objects to JSON and deserializing responses back
    - Setting standard request headers
    - Mapping HTTP error responses to exceptions
- The mental model
  - You describe what you want
    - The URL, the method, the payload
  - RestTemplate handles how to send it and how to receive the response

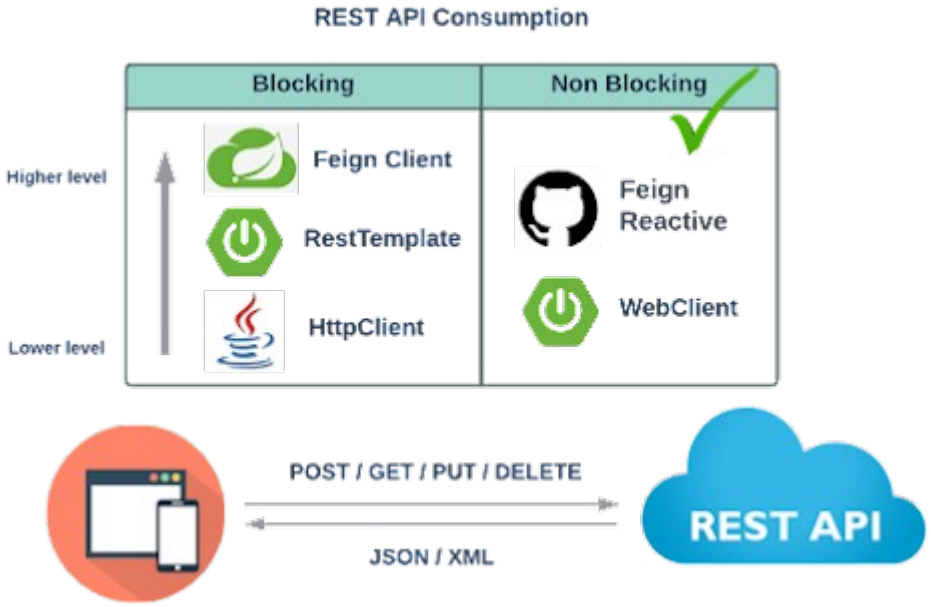


Image Credit: https://javarevisited.blogspot.com/2023/08/difference-between-resttemplate-and.html

RestTemplate Core Concepts

- Three things every RestTemplate call needs
  - The URL: where to send the request with optional URI variables
  - The HTTP method: GET, POST, PUT, DELETE, etc.
  - The expected response type
    - What Java class to deserialize the response into
- RestTemplate handles serialization
  - Uses HttpMessageConverter implementations
  - By default, includes
    - MappingJackson2HttpMessageConverter for JSON
  - Automatically sets Content-Type and Accept headers based on the converters configured
  - No manual JSON parsing required
    - Maps directly to and from your Java classes

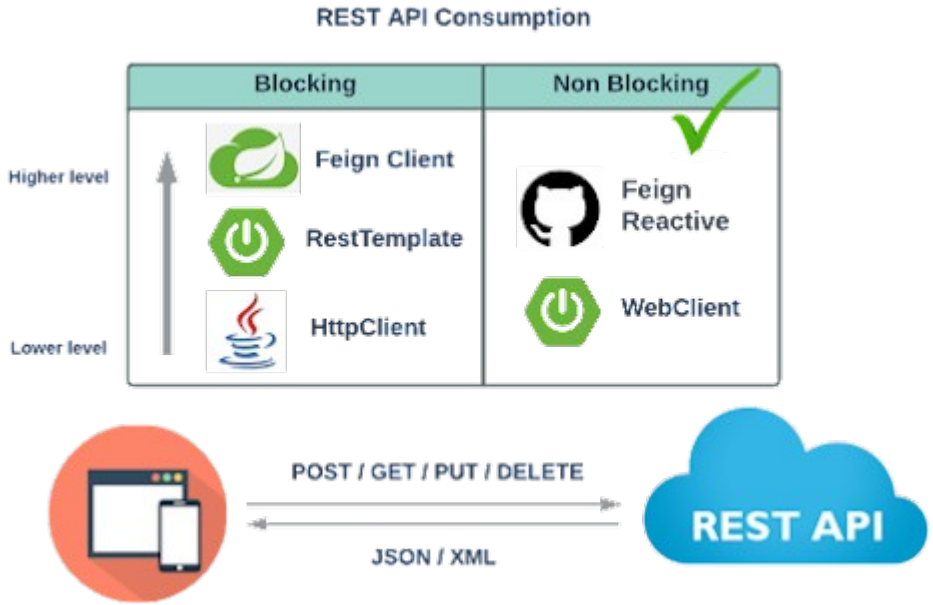


Image Credit: https://javarevisited.blogspot.com/2023/08/difference-between-resttemplate-and.html

RestTemplate Core Concepts

- Error handling
  - Non-2xx responses are automatically converted to exceptions
  - HttpClientErrorException for 4xx responses
  - HttpServerErrorException for 5xx responses
  - Both carry the status code and response body for inspection

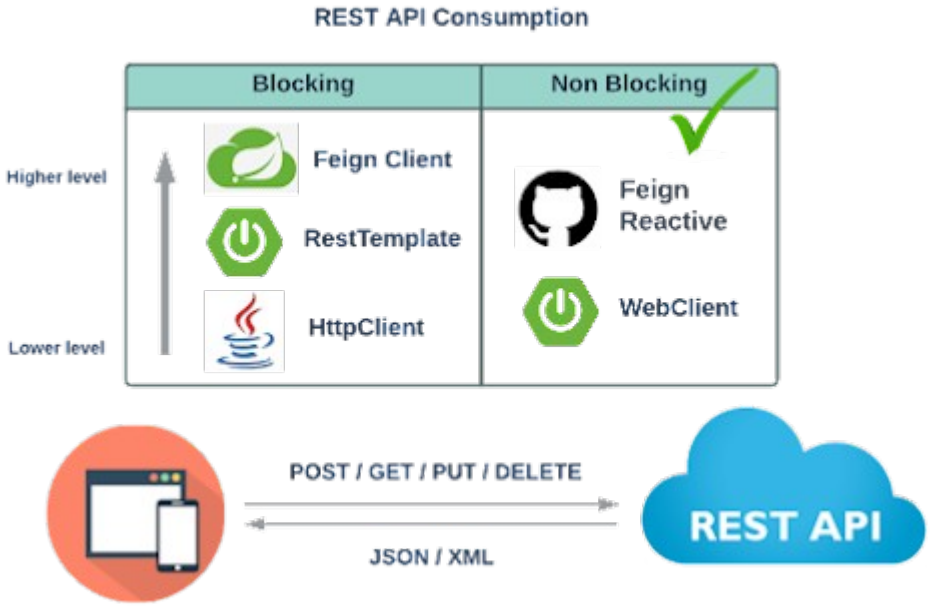


Image Credit: https://javarevisited.blogspot.com/2023/08/difference-between-resttemplate-and.html

Creating an Instance

- Direct instantiation
  - Simple, suitable for basic use
- Using the builder
  - Auto-configured by Spring Boot
    - Picks up any globally registered customizers
    - Timeouts configured in one place
    - rootUri eliminates repetition of the base URL
    - Returns a new instance
    - RestTemplateBuilder itself is safely reusable
  - Always define as a @Bean
    - Makes it injectable across the application
    - Single instance, shared configuration
    - Easy to mock in tests

```
RestTemplate restTemplate = new RestTemplate();
```

```
@Bean
public RestTemplate restTemplate(RestTemplateBuilder builder) {
    return builder
        .rootUri("https://api.example.com")
        .setConnectTimeout(Duration.ofSeconds(3))
        .setReadTimeout(Duration.ofSeconds(10))
        .build();
}
```

Making GET Requests

- GetForObject
  - Simplest form, returns the response body directly
  - Use getForObject when you only need the body
  - Throws and exception on any return code that is not 2xx

```
// Simple GET — response body mapped to a String
String response = restTemplate.getForObject(
    "https://api.example.com/users/{id}",
    String.class,
    userId
);

// GET with a typed response — maps JSON to a Java class automatically
User user = restTemplate.getForObject(
    "https://api.example.com/users/{id}",
    User.class,
    userId
);
```

Making GET Requests

- getForEntity
  - Returns the full HTTP response including status and headers
  - Use getForEntity when you need status codes or response headers
  - In production code, getForEntity is usually the right choice
    - HTTP status codes carry meaning.
  - getForObject is best thought of as a convenience method
    - Good for or straightforward cases where the status code genuinely does not matter to the calling code

```
ResponseEntity<User> response = restTemplate.getForEntity(
    "https://api.example.com/users/{id}",
    User.class,
    userId
);

HttpStatusCode status = response.getStatusCode();   // e.g. 200 OK
User user = response.getBody();                      // deserialized body
HttpHeaders headers = response.getHeaders();         // response headers
```

Making GET Requests

- GetForObject use cases
  - Internal service calls where contracts are well understood and stable
  - Read-only reference data lookups where the only meaningful outcomes are "got the data" or "something went wrong"
  - Prototype or throwaway code where you want minimal boilerplate
  - Cases where the error handler already handles non-2xx responses

```
User user = restTemplate.getForObject(
    "https://internal-user-service/users/{id}",
    User.class,
    userId
);
```

POST, PUT, and DELETE

- postForObject and postForEntity
  - Serialize the request object to JSON automatically using HttpMessageConverter
  - put() and delete() return void
    - There is no return value
    - If you need the response status or headers from a PUT or DELETE, use the exchange() method
- PostForEntity
  - Location header in the response typically contains the URL of the newly created resource

```
// Create a new resource, receive the created resource back
CreateUserRequest request = new CreateUserRequest("alice", "alice@example.com");

User created = restTemplate.postForObject(
    "https://api.example.com/users",
    request,            // serialized to JSON automatically
    User.class
);

// POST and get full response (including Location header)
ResponseEntity<User> response = restTemplate.postForEntity(
    "https://api.example.com/users",
    request,
    User.class
);
URI location = response.getHeaders().getLocation();
```

```
restTemplate.put(
    "https://api.example.com/users/{id}",
    updatedUser,    // serialized to JSON automatically
    userId
);
```

```
restTemplate.delete(
    "https://api.example.com/users/{id}",
    userId
);
```

The exchange Method

- Exchange
  - The general-purpose method for full control
  - Supports any HTTP method
  - Full access to request headers, response headers, and status
  - Required for typed generic responses
    - For example, List<User>

```
ResponseEntity<T> exchange(
    String url,
    HttpMethod method,
    HttpEntity<?> requestEntity,    // headers + body
    Class<T> responseType,
    Object... uriVariables
)
```

```
HttpHeaders headers = new HttpHeaders();
headers.set("X-Api-Key", apiKey);
headers.setAccept(List.of(MediaType.APPLICATION_JSON));

HttpEntity<Void> requestEntity = new HttpEntity<>(headers); // no body for GET

ResponseEntity<User> response = restTemplate.exchange(
    "https://api.example.com/users/{id}",
    HttpMethod.GET,
    requestEntity,
    User.class,
    userId
);
```

```
ResponseEntity<List<User>> response = restTemplate.exchange(
    "https://api.example.com/users",
    HttpMethod.GET,
    HttpEntity.EMPTY,
    new ParameterizedTypeReference<List<User>>() {}  // preserves generic type
);
List<User> users = response.getBody();
```

The exchange Method

- Exchange
  - Most flexible method on RestTemplate
    - Used most often when anything beyond the basics is needed
    - For example, ustom headers, full response inspection, or generic collection types
- ParameterizedTypeReference
  - The solution to Java's type erasure problem
    - At runtime, List<User>.class doesn't exist because Java erases the generic type
    - Captures the full generic type at compile time using an anonymous subclass, allowing RestTemplate to correctly deserialize a JSON array into List<User>

```
HttpHeaders headers = new HttpHeaders();
headers.set("X-Api-Key", apiKey);
headers.setAccept(List.of(MediaType.APPLICATION_JSON));

HttpEntity<Void> requestEntity = new HttpEntity<>(headers); // no body for GET

ResponseEntity<User> response = restTemplate.exchange(
    "https://api.example.com/users/{id}",
    HttpMethod.GET,
    requestEntity,
    User.class,
    userId
);
```

```
ResponseEntity<List<User>> response = restTemplate.exchange(
    "https://api.example.com/users",
    HttpMethod.GET,
    HttpEntity.EMPTY,
    new ParameterizedTypeReference<List<User>>() {}  // preserves generic type
);
List<User> users = response.getBody();
```

The exchange Method

- HttpEntity
  - Container for headers and an optional body
  - For GET and DELETE requests, use HttpEntity.EMPTY
  - Or new HttpEntity<>(headers) with no body argument

```
HttpHeaders headers = new HttpHeaders();
headers.set("X-Api-Key", apiKey);
headers.setAccept(List.of(MediaType.APPLICATION_JSON));

HttpEntity<Void> requestEntity = new HttpEntity<>(headers); // no body for GET

ResponseEntity<User> response = restTemplate.exchange(
    "https://api.example.com/users/{id}",
    HttpMethod.GET,
    requestEntity,
    User.class,
    userId
);
```

```
ResponseEntity<List<User>> response = restTemplate.exchange(
    "https://api.example.com/users",
    HttpMethod.GET,
    HttpEntity.EMPTY,
    new ParameterizedTypeReference<List<User>>() {}  // preserves generic type
);
List<User> users = response.getBody();
```

Authentication Headers

- Manual header attachment
  - Common pattern in older code bases

```
HttpHeaders headers = new HttpHeaders();
headers.setBearerAuth(tokenProvider.getAccessToken());  // sets Authorization: Beare
headers.setContentType(MediaType.APPLICATION_JSON);

HttpEntity<CreateUserRequest> requestEntity =
    new HttpEntity<>(new CreateUserRequest("alice"), headers);

ResponseEntity<User> response = restTemplate.exchange(
    "https://api.example.com/users",
    HttpMethod.POST,
    requestEntity,
    User.class
);
```

Authentication Headers

- ClientHttpRequestInterceptor
  - Sits in a chain and intercepts every request before it is sent
  - Correct pattern for cross-cutting concerns
    - For example, authentication, logging, and correlation ID propagation
  - Interceptors run in registration order
  - Each interceptor calls execution.execute() to pass the request to the next interceptor in the chain

```
public class BearerTokenInterceptor implements ClientHttpRequestInterceptor {

    private final TokenProvider tokenProvider;

    @Override
    public ClientHttpResponse intercept(
            HttpRequest request,
            byte[] body,
            ClientHttpRequestExecution execution) throws IOException {

        request.getHeaders().setBearerAuth(tokenProvider.getAccessToken());
        return execution.execute(request, body);  // proceed with modified request
    }
}
```

```
restTemplate.setInterceptors(List.of(new BearerTokenInterceptor(tokenProvider)));
```

Error Handling

- Default behaviour
  - Exceptions are thrown for non-2xx responses
- The exception hierarchy
  - RestClientException (root)
    - ResourceAccessException: I/O errors, timeouts, connection failures
  - RestClientResponseException: received a response, but it was an error
    - HttpClientErrorException: 4xx errors with inner classes: .NotFound, .Unauthorized, .Forbidden, etc.
    - HttpServerErrorException: 5xx errors

```
try {
    User user = restTemplate.getForObject("/users/{id}", User.class, userId);
} catch (HttpClientErrorException.NotFound e) {
    // 404 — resource does not exist
    log.warn("User not found: {}", userId);
} catch (HttpClientErrorException.Unauthorized e) {
    // 401 — token missing or expired
    log.error("Authentication failed: {}", e.getStatusCode());
} catch (HttpClientErrorException e) {
    // Any other 4xx
    log.error("Client error {}: {}", e.getStatusCode(), e.getResponseBodyAsString())
} catch (HttpServerErrorException e) {
    // Any 5xx
    log.error("Server error {}: {}", e.getStatusCode(), e.getResponseBodyAsString())
} catch (ResourceAccessException e) {
    // Network-level failure — timeout, connection refused
    log.error("Network failure calling upstream: {}", e.getMessage());
}
```

Error Handling

- Custom error handler
  - Replace the default behaviour

```
restTemplate.setErrorHandler(new DefaultResponseErrorHandler() {
    @Override
    public void handleError(ClientHttpResponse response) throws IOException {
        if (response.getStatusCode() == HttpStatus.NOT_FOUND) {
            throw new UserNotFoundException("User not found");
        }
        super.handleError(response);  // default handling for everything else
    }
});
```

Summary

- What RestTemplate does well
  - Simple, readable API for straightforward synchronous HTTP calls
  - Automatic JSON serialization/deserialization with no boilerplate
  - Interceptor chain for cross-cutting concerns
  - Clear exception hierarchy for error handling
  - Extensive existing documentation and community knowledge
- Its fundamental limitations
  - Blocking only, one thread is held for the duration of every call
  - No built-in retry or resilience primitives
  - No streaming support
  - No HTTP/2 support
  - In maintenance mode, no new features

WebClient

WebClient

- WebClient
  - Spring's current HTTP client
    - Introduced in Spring 5 as part of Spring WebFlux
  - Supports both blocking and non-blocking execution models
  - Fluent, composable API
  - First-class support for reactive streams
- Actively maintained
  - Receiving new features

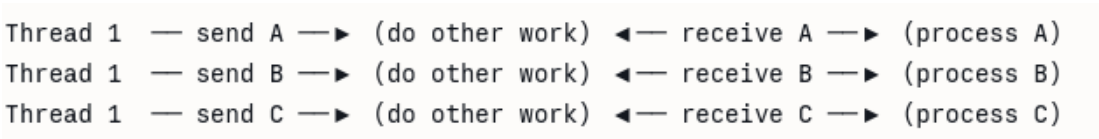
Capability	RestTemplate	WebClient
Synchronous calls	yes	yes
Non-blocking / reactive	no	yes
Streaming responses	no	yes
HTTP/2	no	yes
Filter chain (interceptors)	Limited	Full support
Active development	no	yes

WebClient

- Threading problem that RestTemplate cannot solve
- Every outbound HTTP call blocks the calling thread until the response arrives
  - A thread sitting idle waiting for a network response is wasted capacity
  - Under load, thread pools exhaust and the application stops accepting work
  - During that 200ms wait, the thread is doing no useful work
    - It is consuming memory, each JVM thread ~1MB stack by default
    - It is holding a slot in the thread pool that another request needs
    - The upstream's response time directly caps your application's throughput

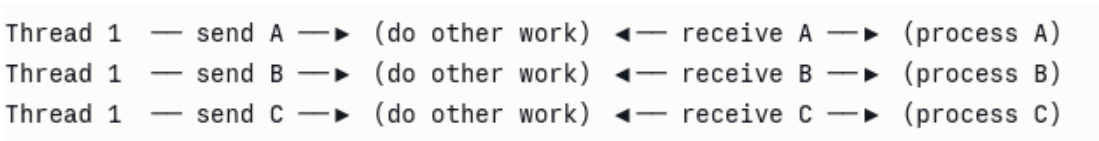
WebClient

- Non-blocking alternative
  - The thread initiates the request and immediately moves on
  - The OS notifies the application when the response arrives
  - A small number of threads can manage a large number of in-flight requests
  - One thread handles multiple in-flight requests
  - CPU used only when there is work to do
  - Thread count is no longer the throughput ceiling



WebClient

- Key shift
  - From "thread per connection"
  - To "event-driven callbacks"
  - The OS kernel manages the actual I/O wait
  - The application is notified when data is ready to process.
- Uses
  - NIO, Java's Non-blocking I/O
  - And event loops to use WebClient effectively



Mono and Flux

- WebClient does the data directly
  - It returns a description of future data
  - Mono<T>
    - Represents zero or one result arriving asynchronously
  - Flux<T>
    - Represents zero or many results arriving asynchronously
  - Neither executes anything until something subscribes to it
    - Nothing happens until the pipeline is executed
    - .block(): subscribe and wait synchronously, acceptable in Spring MVC
    - .subscribe(): subscribe and continue without waiting, reactive applications

```
WebClient call
    |
    ▼
Mono<User>  ⟵ "A User will arrive here eventually"
    |
    ├── .map(user -> transform(user))      ⟵ "When it arrives, transform it"
    |
    ├── .filter(user -> user.isActive())   ⟵ "Only proceed if active"
    |
    ├── .onErrorReturn(fallbackUser)       ⟵ "If something fails, use this"
    |
    └── .block()                           ⟵ "Execute now and wait for result"
```

Mono and Flux

- The return value is not the data
  - It is a recipe for obtaining the data
  - Called a "cold publisher" in reactive programming
  - Nothing happens until someone subscribes.
- The pipeline
  - Each step in the chain
    - Is declared in terms of what to do when data arrives
    - Not what to do right now
    - This is what allows the non-blocking execution model to work
- Using WebClient in a Spring MVC non-reactive app
  - .block() is the bridge
  - It subscribes to the Mono, waits for the result, and returns it as a plain Java object
  - This is entirely acceptable and commonly used

```
WebClient call
    |
    ▼
Mono<User>  ⟵ "A User will arrive here eventually"
    |
    ├── .map(user -> transform(user))      ⟵ "When it arrives, transform it"
    |
    ├── .filter(user -> user.isActive())   ⟵ "Only proceed if active"
    |
    ├── .onErrorReturn(fallbackUser)       ⟵ "If something fails, use this"
    |
    └── .block()                           ⟵ "Execute now and wait for result"
```

The Filter Chain

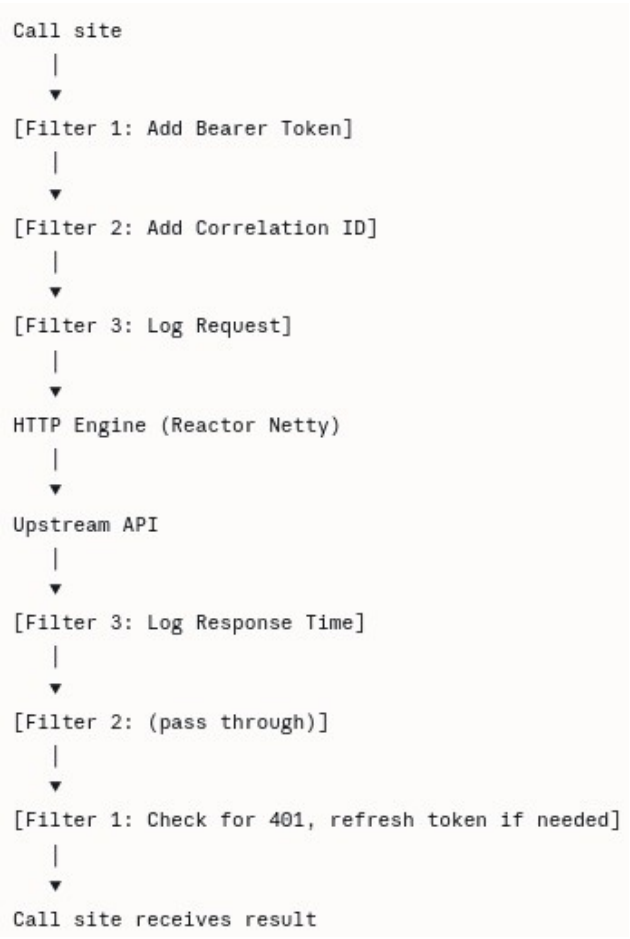
- WebClient processes every request through a chain of filters
  - Filters are applied in order before the request is sent
  - Filters can also intercept and inspect the response
  - Each filter decides whether to modify the request, pass it on, or short-circuit
- Why the filter chain matters
  - Everything is done once and in one place
    - Authentication (add Bearer token)
    - Logging (record method, URL, timing)
    - Correlation ID propagation
    - Retry logic
  - All call sites are clean because they only express business intent

```
// Without filters — every call site carries the burden
client.get()
    .uri("/users/{id}", id)
    .header("Authorization", "Bearer " + getToken())
    .header("X-Correlation-Id", MDC.get("correlationId"))
    .retrieve()
    .bodyToMono(User.class)
    .block();
```

```
client.get()
    .uri("/users/{id}", id)
    .retrieve()
    .bodyToMono(User.class)
    .block();
```

The Filter Chain

- Similar to the filter chain in Spring Security
  - Both are middleware pipeline pattern
  - Composability: each filter has one job and has no knowledge of the others



The Filter Chain

- A WebClient instance is configured for one upstream service
  - baseUrl is fixed at construction time
  - Filters are registered at construction time
  - Authentication configuration is fixed at construction time
  - The instance is immutable after construction to be thread-safe by design
    - Each upstream may require different credentials
    - Each upstream may have different timeout requirements
    - Circuit breakers are configured per upstream
    - Logging and tracing can identify which upstream is involved

```
@Bean
public WebClient userServiceClient(...) {
    return WebClient.builder()
        .baseUrl("${services.user.base-url}")
        .filter(oauth2Filter("user-service"))
        .filter(loggingFilter("user-service"))
        .build();
}

@Bean
public WebClient paymentServiceClient(...) {
    return WebClient.builder()
        .baseUrl("${services.payment.base-url}")
        .filter(oauth2Filter("payment-service"))
        .filter(loggingFilter("payment-service"))
        .build();
}
```

Controlling Serialization

- Codecs are the serialization/deserialization layer in WebClient
  - Handle conversion between Java objects and HTTP request/response bodies
  - Configured via ExchangeStrategies
  - Default codecs include Jackson for JSON, String, byte arrays, and form data
- Why you might need to configure codecs
  - The default buffer size for response bodies is 256KB
  - Responses larger than this will throw DataBufferLimitException
  - In production, APIs sometimes return large payloads like reports, bulk data, search results

```
@Bean
public WebClient myClient(WebClient.Builder builder) {
    return builder
        .baseUrl("${api.base-url}")
        .exchangeStrategies(ExchangeStrategies.builder()
            .codecs(configurer ->
                configurer.defaultCodecs()
                    .maxInMemorySize(2 * 1024 * 1024)  // 2MB
            )
            .build())
        .build();
}
```

URI Construction

- Why URI template variables matter
  - Values are automatically percent-encoded and special characters are handled correctly
  - Resistant to path traversal
    - For example, {id} is treated as a single segment, not parsed
  - Clearly separates the URL structure from the variable values
  - Readable and maintainable
- What the UriBuilder gives you
  - Fluent construction of query parameters without manual encoding
  - Conditional parameter inclusion
  - Testable URL construction logic

```
// ❌ Fragile – encoding issues, injection risk, hard to read
String url = baseUrl + "/users/" + userId + "?status=" + status;

// ✅ Use URI template variables
client.get()
    .uri("/users/{id}", userId)
    ...

// ✅ Use the UriBuilder for complex URLs
client.get()
    .uri(uriBuilder -> uriBuilder
        .path("/users/{id}/transactions")
        .queryParam("status", status)
        .queryParam("from", fromDate)
        .queryParam("limit", 50)
        .build(userId))
```

Headers and Request Attributes

- Setting headers that apply to every request

```
WebClient client = WebClient.builder()
    .baseUrl("${api.base-url}")
    .defaultHeader(HttpHeaders.ACCEPT, MediaType.APPLICATION_JSON_VALUE)
    .defaultHeader(HttpHeaders.CONTENT_TYPE, MediaType.APPLICATION_JSON_VALUE)
    .defaultHeader("X-Api-Version", "2024-01")
    .build();
```

- Overriding defaults on a per-request basis

```
// Default Accept is application/json, but this call needs XML
client.get()
    .uri("/reports/{id}", reportId)
    .accept(MediaType.APPLICATION_XML)   // overrides the default for this call
    .retrieve()
    .bodyToMono(String.class)
    .block();
```

Headers and Request Attributes

- Request attributes
  - Passing context through the filter chain

```
// Set an attribute on the request
client.get()
    .uri("/users/{id}", userId)
    .attribute("retryCount", 0)
    .retrieve()
    .bodyToMono(User.class)
    .block();

// Read the attribute inside a filter
ExchangeFilterFunction.ofRequestProcessor(request -> {
    Integer retries = (Integer) request.attribute("retryCount").orElse(0);
    // use the attribute value
    return Mono.just(request);
});
```

Response

- Two ways to process the response
- retrieve()
  - The standard, safe path
  - Spring automatically reads and releases the response body
  - Non-2xx responses automatically throw WebClientResponseException
  - No risk of connection leaks

```
User user = client.get()
    .uri("/users/{id}", userId)
    .retrieve()                          // Spring manages the response lifecycle
    .bodyToMono(User.class)
    .block();
```

Response

- Two ways to process the response
- exchangeToMono()
  - Full control, full responsibility
  - You receive the full ClientResponse
    - Includes status, headers, body
  - You must consume the body in every code path
    - Failing to do so leaks the connection
  - Use only when you need header inspection or conditional body handling

```
User user = client.get()
    .uri("/users/{id}", userId)
    .exchangeToMono(response -> {
        if (response.statusCode().is2xxSuccessful()) {
            return response.bodyToMono(User.class);
        } else {
            return response.createError();  // you must handle every branch
        }
    })
    .block();
```

Filter Ordering

- Filters execute in registration order
- Practical ordering rules
  - Correlation ID first
    - Everything logged after it will carry the ID
  - Token attachment before logging
    - Logs will show auth is present
  - Error-handling filters last
    - They see the final response

Error Handling

- Why map to domain exceptions?
  - Your service layer should speak your domain, not HTTP
  - Callers handle
    - UserNotFoundException
    - Not WebClientResponseException
  - Consistent error handling regardless of which HTTP client is used

```
User user = client.get()
    .uri("/users/{id}", userId)
    .retrieve()
    .onStatus(
        HttpStatusCode::is4xxClientError,
        response -> response.bodyToMono(ApiErrorResponse.class)
            .map(error -> switch (response.statusCode().value()) {
                case 404 -> new UserNotFoundException(userId);
                case 403 -> new AccessDeniedException("Insufficient permissions");
                case 429 -> new RateLimitException("Rate limit exceeded");
                default  -> new ClientException(error.getMessage());
            })
    )
    .onStatus(
        HttpStatusCode::is5xxServerError,
        response -> response.bodyToMono(String.class)
            .map(body -> new UpstreamServiceException(
                "Upstream error: " + response.statusCode()))
    )
    .bodyToMono(User.class)
    .block();
```

Responses with No Body

- Not all responses carry a body
  - 204 No Content
    - The operation succeeded, nothing to return
  - 201 Created
    - The resource was created; the location is in the header
  - 200 OK on DELETE
    - Sometimes returned instead of 204
  - Sidebar shows
    - 204 response
    - 201 response
    - Accessing response headers alongside the body

```
// bodyToMono(Void.class) correctly handles empty bodies
client.delete()
    .uri("/users/{id}", userId)
    .retrieve()
    .bodyToMono(Void.class)
    .block();
```

```
URI createdLocation = client.post()
    .uri("/users")
    .bodyValue(createRequest)
    .retrieve()
    .toBodilessEntity()             // returns ResponseEntity<Void>
    .map(response -> response.getHeaders().getLocation())
    .block();
```

```
ResponseEntity<User> response = client.get()
    .uri("/users/{id}", userId)
    .retrieve()
    .toEntity(User.class)           // returns ResponseEntity<User>
    .block();

User user = response.getBody();
String etag = response.getHeaders().getETag();
```

Timeouts

- Two levels of timeout control
- Level 1 HTTP engine timeout
  - Configured at WebClient construction
  - Applies to all requests through this WebClient instance
  - Connection timeout, response timeout
  - Covered in the resilience section
- Level 2 Reactive timeout
  - Applied per call

```
User user = client.get()
    .uri("/users/{id}", userId)
    .retrieve()
    .bodyToMono(User.class)
    .timeout(Duration.ofSeconds(5))    // this specific call must complete in 5s
    .block();
```

Timeouts

- When to use per-call timeouts
  - Operations with tighter SLAs than the default
    - For example, user-facing real-time calls
  - Operations that are known to be consistently fast
    - Enforce the expectation
- Distinguishing between acceptable and unacceptable latency for different endpoints

```
User user = client.get()
    .uri("/users/{id}", userId)
    .retrieve()
    .bodyToMono(User.class)
    .timeout(Duration.ofSeconds(5))    // this specific call must complete in 5s
    .block();
```

Timeouts

- The two-level timeout model gives fine-grained control
  - The HTTP engine timeout is the safety net
    - Catches cases where the TCP connection or response delivery takes too long at the network level
  - The reactive .timeout() is the business-level timeout
    - it enforces an end-to-end time budget for a specific operation.
  - In practice
    - Set conservative timeouts at the HTTP engine level for all requests
    - Tighten them with .timeout() for specific high-priority or latency-sensitive operations.

```
User user = client.get()
    .uri("/users/{id}", userId)
    .retrieve()
    .bodyToMono(User.class)
    .timeout(Duration.ofSeconds(5))    // this specific call must complete in 5s
    .block();
```

Timeouts

- The two-level timeout model gives fine-grained control
  - The HTTP engine timeout is the safety net
    - Catches cases where the TCP connection or response delivery takes too long at the network level
  - The reactive .timeout() is the business-level timeout
    - it enforces an end-to-end time budget for a specific operation.
  - In practice
    - Set conservative timeouts at the HTTP engine level for all requests
    - Tighten them with .timeout() for specific high-priority or latency-sensitive operations.

```
User user = client.get()
    .uri("/users/{id}", userId)
    .retrieve()
    .bodyToMono(User.class)
    .timeout(Duration.ofSeconds(5))    // this specific call must complete in 5s
    .block();
```

The WebClient Mental Model

- Three foundational ideas to carry into production
  - 1. WebClient is a pipeline, not a function call
    - You declare what to do with data when it arrives
    - he pipeline executes only when subscribed to (.block() in Spring MVC)
    - Each step in the pipeline is a transformation or side effect
  - 2. Filters are where cross-cutting concerns live
    - Authentication, logging, tracing, error mapping all belong in filters
    - Call sites express only business intent
    - Filters are composable and independently testable
  - 3. One WebClient bean per upstream
    - Configuration is fixed at construction time
    - Each upstream has its own auth, timeouts, and circuit breaker
    - Thread-safe, hared freely across the application

The WebClient Mental Model

- The production checklist
  - Timeouts configured at the HTTP engine level
  - Bearer token attached via filter, not at call sites
  - Secrets in environment variables, not in source code
  - onStatus handlers map HTTP errors to domain exceptions
  - Retry configured for idempotent operations only
  - Circuit breaker configured for critical dependencies
  - Safe logging filter — no Authorization header values logged
  - Correlation ID propagated on all outbound calls
  - Tests written with MockWebServer or WireMock

Bearer Tokens

Bearer Tokens

- Bearer token
  - A credential that grants access to a resource
    - Anyone "bearing" the token can use it
    - There is no binding to a specific client identity
  - Almost always a signed JWT in OAuth2 contexts
  - Transmitted in the HTTP Authorization header
- Standard header format RFC 6750

Authorization: Bearer eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...

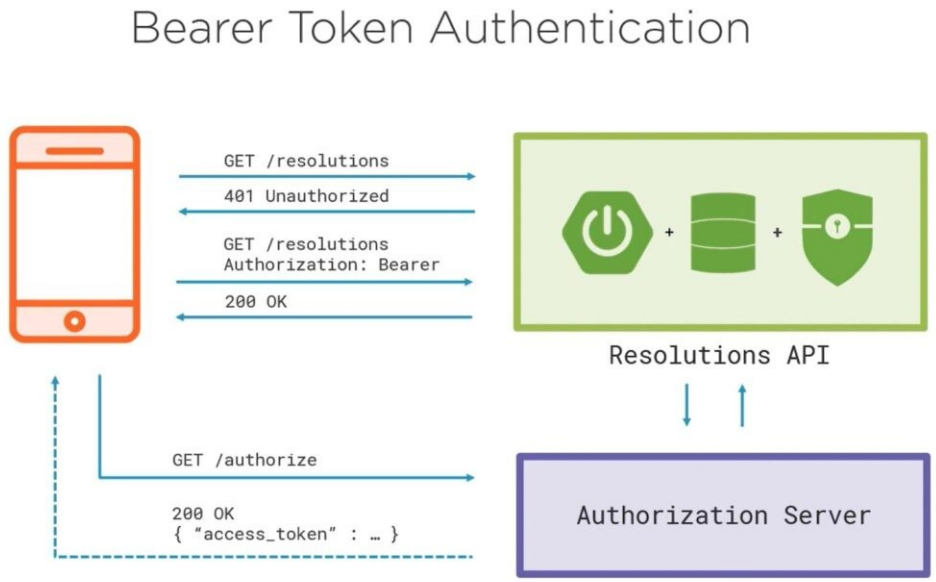


Image Credit: https://www.tokendesk.io/how-to-get-bearer-token/

Bearer Tokens

- Why this matters for outbound calls
  - You are asserting your service's identity to an upstream
  - The upstream validates the token's signature, expiry, and scopes
  - Mishandling the token is a security failure
    - For example, logging it, caching it incorrectly, sending it over HTTP
- Golden rules
  - Never log the full token value
  - Never send tokens over plain HTTP
  - Validate token expiry before use
    - Do not wait for a 401
- Treat tokens as credentials, not as data

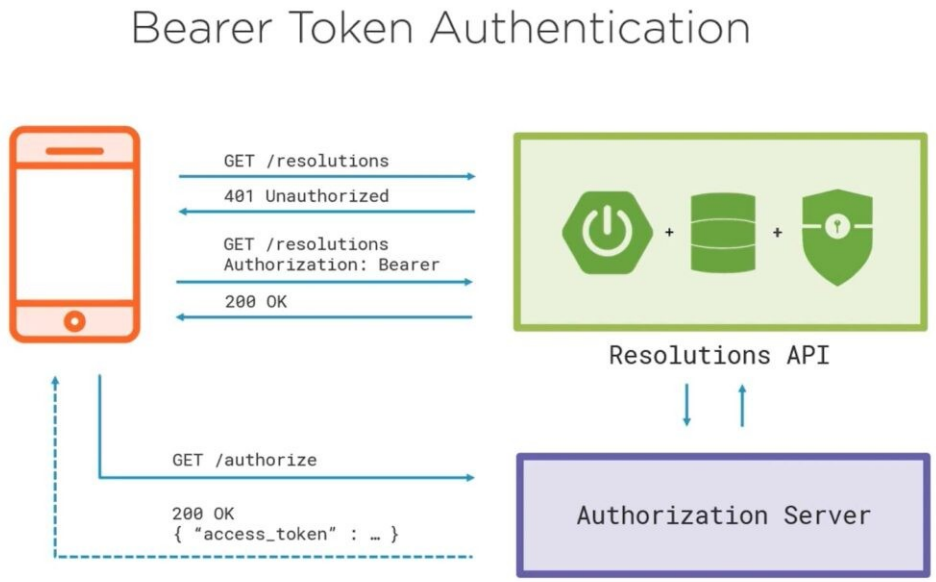


Image Credit: https://www.tokendesk.io/how-to-get-bearer-token/

Attaching Tokens

- Wrong: Inline at every call site
  - Token retrieval logic duplicated across the codebase
  - No consistent handling — each developer does it differently
  - Hard to audit whether all calls are authenticated
  - Token refresh logic has no central home

```
// Scattered, hard to audit, easy to miss
client.get()
    .uri("/accounts")
    .header("Authorization", "Bearer " + getTokenSomehow())
    .retrieve()
    .bodyToMono(AccountList.class)
    .block();
```

Attaching Tokens

- Right: Use a WebClient Filter
  - One place to manage token attachment
  - One place to manage token refresh
  - Fully testable in isolation
  - Invisible to call sites

```
WebClient client = WebClient.builder()
    .baseUrl(baseUrl)
    .filter(bearerTokenFilter())
    .build();
```

Bearer Token Filter

- ExchangeFilterFunction
  - The WebClient filter contract
  - Decouples token acquisition from token usage
  - Implementations can fetch, cache, or refresh as needed
  - Easy to mock in tests

```
public ExchangeFilterFunction bearerTokenFilter(TokenProvider tokenProvider) {
    return ExchangeFilterFunction.ofRequestProcessor(request ->
        Mono.fromCallable(tokenProvider::getAccessToken)
            .map(token ->
                ClientRequest.from(request)
                    .header(HttpHeaders.AUTHORIZATION, "Bearer " + token)
                    .build()
            )
    );
}
```

```
public interface TokenProvider {
    String getAccessToken();
}
```

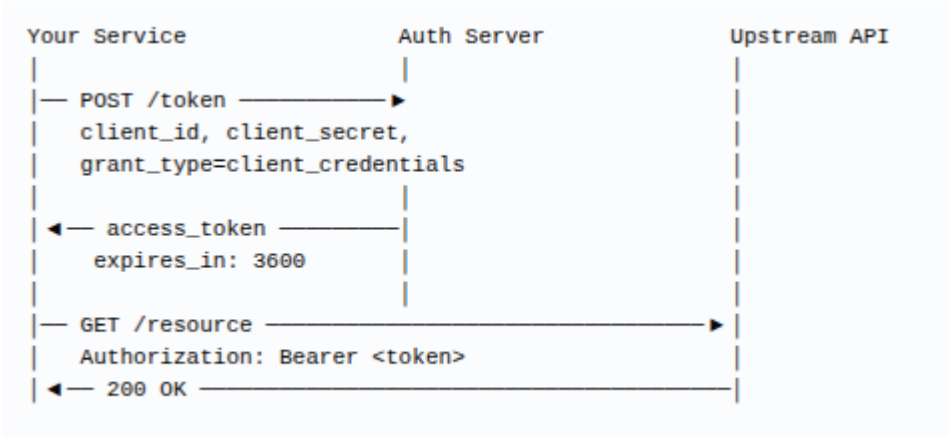
Bearer Token Filter

- Mechanics
  - ofRequestProcessor
    - Transforms the outgoing request before it is sent
  - ClientRequest.from(request)
    - Copies the existing request and lets you add headers
  - Mono.fromCallable()
    - Wraps the potentially blocking token retrieval in a reactive-friendly way
  - Call site → WebClient → Filter (adds token) → HTTP engine → Upstream API

```
@Bean
public WebClient partnerApiClient(TokenProvider tokenProvider) {
    return WebClient.builder()
        .baseUrl("${partner.api.base-url}")
        .filter(bearerTokenFilter(tokenProvider))
        .build();
}
```

Token Acquisition

- The standard machine-to-machine OAuth2 flow
- Spring Security automates this entirely
  - spring-boot-starter-oauth2-client dependency
  - spring.security.oauth2.client.* configuration
  - ServletOAuth2AuthorizedClientExchangeFilterFunction
    - Handles token acquisition and attachment automatically



Token Acquisition

- Spring Security OAuth2 Client: Automated Token Management
- Dependencies and configurations

```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-oauth2-client</artifactId>
</dependency>
```

```yaml
spring:
  security:
    oauth2:
      client:
        registration:
          partner-api:
            client-id: ${PARTNER_CLIENT_ID}
            client-secret: ${PARTNER_CLIENT_SECRET}
            authorization-grant-type: client_credentials
            scope: read:accounts,write:transactions
        provider:
          partner-api:
            token-uri: https://auth.partner.com/oauth/token
```

Token Acquisition

- Wiring the filter

```
@Bean
public WebClient partnerApiClient(OAuth2AuthorizedClientManager manager) {
    var oauth2Filter = new ServletOAuth2AuthorizedClientExchangeFilterFunction(manager);
    oauth2Filter.setDefaultClientRegistrationId("partner-api");

    return WebClient.builder()
            .baseUrl("${partner.api.base-url}")
            .apply(oauth2Filter.oauth2Configuration())
            .build();
}
```

Token Acquisition

- Wiring the filter
- What Spring Security does
  - On first request, it detects there is no valid token cached
  - It calls the token endpoint with the client credentials
  - It caches the token
    - In memory by default, or a custo OAuth2AuthorizedClientRepository)
  - It attaches the token to the outbound request
  - On subsequent requests, it uses the cached token until it is close to expiry
  - When expiry approaches, it automatically re-fetches

```
@Bean
public WebClient partnerApiClient(OAuth2AuthorizedClientManager manager) {
    var oauth2Filter = new ServletOAuth2AuthorizedClientExchangeFilterFunction(manager);
    oauth2Filter.setDefaultClientRegistrationId("partner-api");

    return WebClient.builder()
            .baseUrl("${partner.api.base-url}")
            .apply(oauth2Filter.oauth2Configuration())
            .build();
}
```

Secure Configuration

- The baseline rule
  - secrets never appear in source code
  - No credentials in application.properties or application.yml committed to version control
  - No hardcoded strings in Java files
- Common mistakes
  - application-local.yml committed to git
  - Secrets in docker-compose.yml in the repository
  - Test files containing real credentials

Pattern	Mechanism	Suitable For
Environment variables	`${ENV_VAR_NAME}` in config	Containers, CI/CD pipelines
Spring Cloud Config	Centralized config server	Multi-service environments
HashiCorp Vault	Dynamic secrets, rotation	Enterprise, regulated environments
AWS/Azure/GCP Secret Manager	Cloud-native secret stores	Cloud deployments

Secure Configuration

- The baseline rule
  - secrets never appear in source code
  - No credentials in application.properties or application.yml committed to version control
  - No hardcoded strings in Java files
- Common mistakes
  - application-local.yml committed to git
  - Secrets in docker-compose.yml in the repository
  - Test files containing real credentials

Pattern	Mechanism	Suitable For
Environment variables	`${ENV_VAR_NAME}` in config	Containers, CI/CD pipelines
Spring Cloud Config	Centralized config server	Multi-service environments
HashiCorp Vault	Dynamic secrets, rotation	Enterprise, regulated environments
AWS/Azure/GCP Secret Manager	Cloud-native secret stores	Cloud deployments

Resilience Patterns

Resilience Patterns

- Upstream services will fail
  - This is a realistic design assumption
  - Networks drop packets and introduce latency spikes
  - Upstream services deploy and restart
  - Load spikes cause temporary throttling
    - 429 Too Many Requests
  - Infrastructure changes cause transient DNS and routing issues
  - Without resilience, a single upstream failure becomes your failure
    - Thread exhaustion from hung connections
    - Cascading failures across downstream services
    - Data loss from retried operations landing only once
    - Users see errors from transient conditions that would have resolved

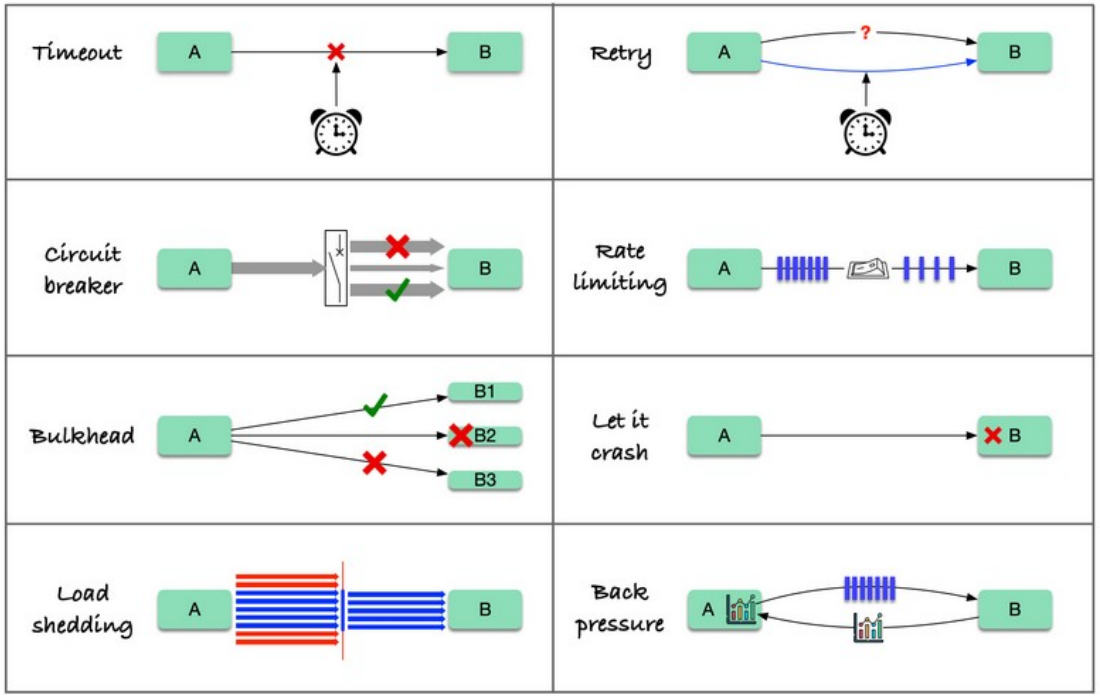


Image Credit: https://bytebytego.com/guides/guides/resiliency-patterns/

Resilience Patterns

- The four essential resilience patterns
  - Timeout
    - Bound how long you wait
  - Retry
    - recover from transient failures automatically
  - Backoff
    - Avoid hammering a struggling upstream
  - Circuit Breaker
    - Stop calling a failing upstream entirely

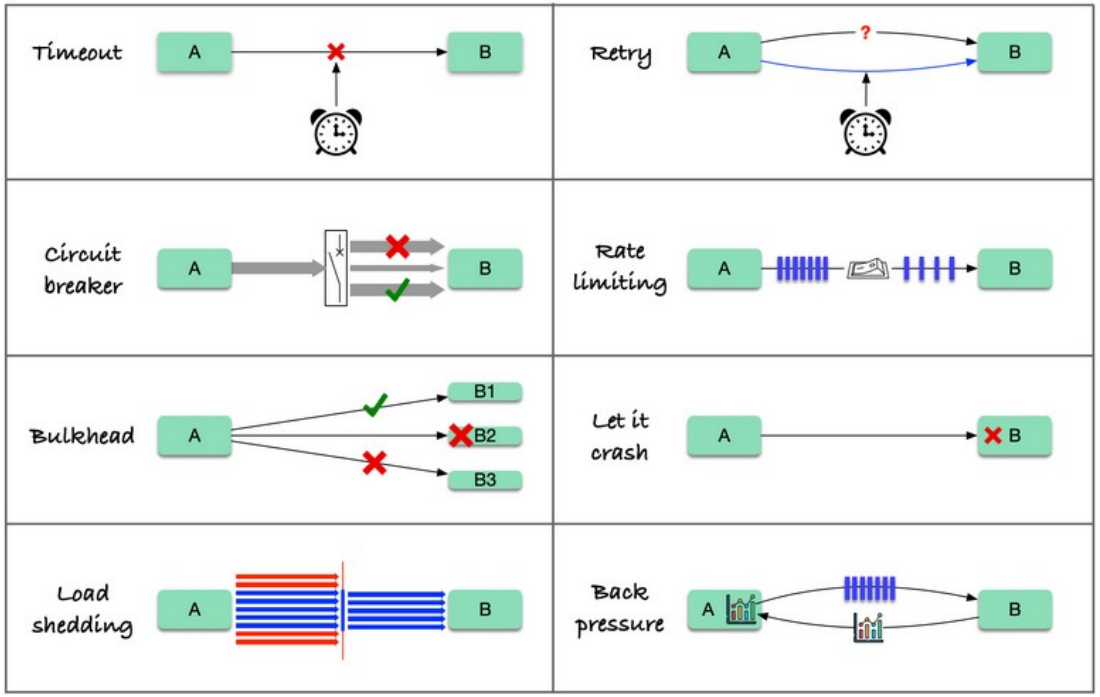


Image Credit: https://bytebytego.com/guides/guides/resiliency-patterns/

Resilience Patterns

- Assumptions developers incorrectly make about networks
  - The network is reliable
  - Latency is zero
  - Bandwidth is infinite
  - The network is secure
  - Topology doesn't change
  - There is one administrator
  - Transport cost is zero
  - The network is homogeneous

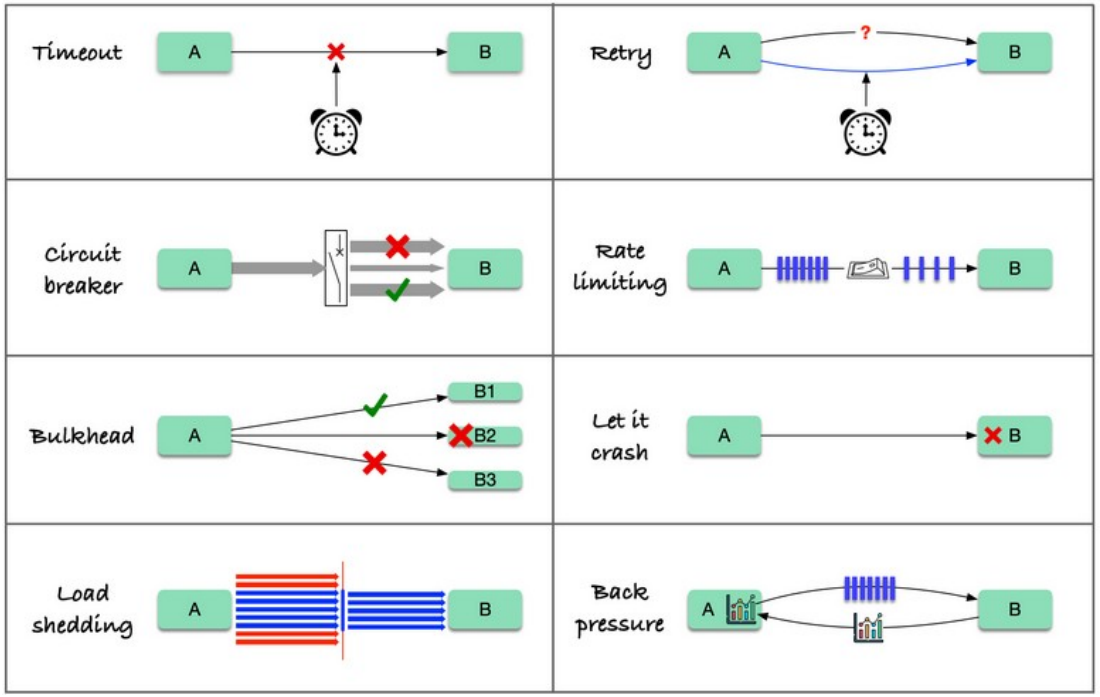


Image Credit: https://bytebytego.com/guides/guides/resiliency-patterns/

Time Out

- Definition
  - A hard limit on how long a call is allowed to take
  - If exceeded, the call is cancelled and an error is raised
- Why you need it
  - Without timeouts, a slow upstream holds your thread indefinitely
  - Thread pools are finite
    - Exhaustion causes application-wide failure
  - Timeouts are the minimum viable resilience control

Type	What it bounds
Connection timeout	Time to establish TCP connection
Read timeout	Time waiting for data after connection established
Response timeout	Total time for full response to arrive
Request timeout	Total time budget including retries

```
HttpClient httpClient = HttpClient.create()
    .option(ChannelOption.CONNECT_TIMEOUT_MILLIS, 3_000)   // 3s connect
    .responseTimeout(Duration.ofSeconds(10));               // 10s total response

WebClient client = WebClient.builder()
    .clientConnector(new ReactorClientHttpConnector(httpClient))
    .build();
```

Retry

- Definition
  - Automatically re-issue a failed request a limited number of times
  - Transparent to the call site
  - When retry is safe: idempotent operations
    - GET, HEAD, OPTIONS, DELETE, safe to retry
    - PUT with full resource representation, safe to retry
  - POST is not safe by default
    - Retrying may create duplicate records

```
client.get()
    .uri("/accounts/{id}", accountId)
    .retrieve()
    .bodyToMono(Account.class)
    .retryWhen(Retry.max(3)
        .filter(ex -> ex instanceof WebClientResponseException.ServiceUnavailable));
```

Retry

- When NOT to retry
  - 400 Bad Request
    - Your request is malformed; retrying will not help
  - 401 Unauthorized
    - Retrying without a new token will not help
  - 403 Forbidden
    - Retrying with the same token will not help
  - 404 Not Found
    - The resource does not exist; retrying will not help
  - Only retry on
    - Network errors, 408, 429, 500, 502, 503, 504

```
client.get()
    .uri("/accounts/{id}", accountId)
    .retrieve()
    .bodyToMono(Account.class)
    .retryWhen(Retry.max(3)
        .filter(ex -> ex instanceof WebClientResponseException.ServiceUnavailable));
```

Exponential Backoff with Jitter

- The problem with naive retry
  - Request fails at t=0
  - Retry at t=1s → fails
  - Retry at t=2s → fails
  - Retry at t=3s → fails
- If 1,000 clients all fail at the same time, they all retry at the same time
  - This creates a retry storm referred to as “the thundering herd”
  - Overwhelms an already struggling upstream

```
client.get()
    .uri("/accounts/{id}", accountId)
    .retrieve()
    .bodyToMono(Account.class)
    .retryWhen(Retry.max(3)
        .filter(ex -> ex instanceof WebClientResponseException.ServiceUnavailable));
```

Exponential Backoff with Jitter

- Exponential backoff
  - Each retry waits longer than the previous
    - 1s → 2s → 4s → 8s
  - Spreads load over time as retries cascade
- Jitter
  - Randomize the backoff window
    - Adds random variation to retry delays
  - Prevents synchronized retry storms across multiple client instances
  - AWS and Google both recommend jitter in their resilience guidance

```
.retryWhen(
    Retry.backoff(3, Duration.ofMillis(500))   // 3 retries, 500ms base
         .maxBackoff(Duration.ofSeconds(10))    // cap at 10s
         .jitter(0.5)                           // ±50% jitter
         .filter(this::isRetryable)
)
```

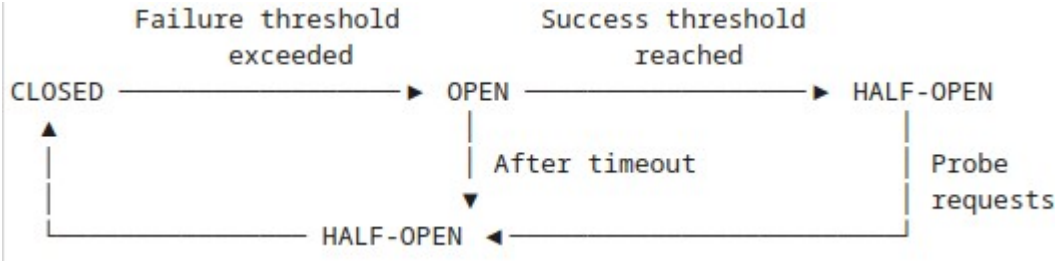
Exponential Backoff with Jitter

- The thundering herd problem is a real production phenomenon
  - When a dependency goes down and comes back up
  - A wave of synchronized retries from all clients can immediately push it back down
- With pure exponential backoff
  - All clients that saw the failure at t=0 will retry at t=1s, t=2s, t=4s simultaneously
  - With full jitter, each client independently picks a random time in the window, spreading the load

```
.retryWhen(
    Retry.backoff(3, Duration.ofMillis(500))   // 3 retries, 500ms base
        .maxBackoff(Duration.ofSeconds(10))     // cap at 10s
        .jitter(0.5)                            // ±50% jitter
        .filter(this::isRetryable)
)
```

Circuit Breaker

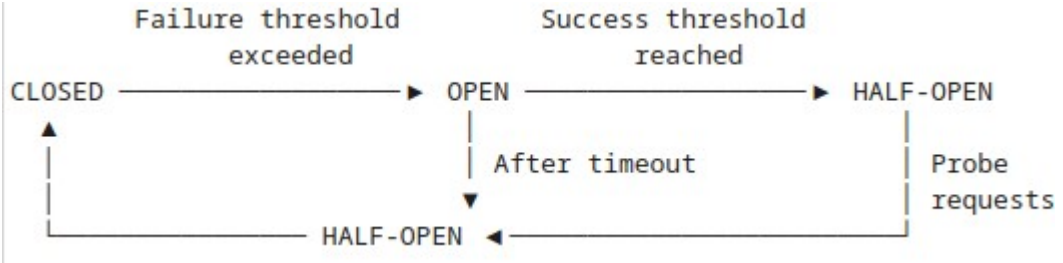
- The problem retry alone does not solve
  - An upstream is down for 2 minutes
  - Every request retries 3 times with backoff
  - Threads are tied up on requests that will never succeed
  - Your service degrades under load from its own retry traffic



State	Behaviour
Closed	Requests flow normally; failures are counted
Open	Requests fail immediately (fast fail) — no upstream calls made
Half-Open	A limited number of probe requests are allowed through

Circuit Breaker

- The software circuit breaker trips to protect your service from an overloaded upstream
- Circuit breakers protect both your service and the upstream
- By stopping retry traffic when the upstream is clearly struggling, you give it a chance to recover.



State	Behaviour
Closed	Requests flow normally; failures are counted
Open	Requests fail immediately (fast fail) — no upstream calls made
Half-Open	A limited number of probe requests are allowed through

Circuit Breaker

- Adding Resilience4j config
  - Successor to Netflix Hystrix
  - Current industry standard for resilience patterns in Java/Spring applications
  - It integrates natively with Spring Boot via auto-configuration
  - Configuration parameters:
    - sliding-window-size: 10 - The circuit evaluates the last 10 calls
    - failure-rate-threshold: 50 - If 5 or more of the last 10 calls fail, open the circuit
    - wait-duration-in-open-state: 30s - wait 30 seconds before allowing probe requests

```xml
<dependency>
    <groupId>io.github.resilience4j</groupId>
    <artifactId>resilience4j-spring-boot3</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-aop</artifactId>
</dependency>
```

```yaml
resilience4j:
  circuitbreaker:
    instances:
      partnerApi:
        sliding-window-size: 10
        failure-rate-threshold: 50        # Open if >50% of last 10 calls fail
        wait-duration-in-open-state: 30s  # Stay open for 30s before probing
        permitted-calls-in-half-open-state: 3
```

Circuit Breaker

- Adding Resilience4j config
  - The fallback method is critical
  - Simply failing fast without a fallback is only half the pattern
  - Fallback options
    - Return cached data
    - Return a degraded response
    - Return a default
    - Queue the operation for later.

```
@CircuitBreaker(name = "partnerApi", fallbackMethod = "getAccountFallback")
public Account getAccount(String accountId) {
    return partnerApiClient.get()
        .uri("/accounts/{id}", accountId)
        .retrieve()
        .bodyToMono(Account.class)
        .block();
}

private Account getAccountFallback(String accountId, Exception ex) {
    log.warn("Circuit open for partnerApi, returning cached data for {}", accountId)
    return cache.getLastKnownAccount(accountId);
}
```

The Resilience Stack

- Ordering matters
  - Circuit breaker wraps the entire operation including retries
  - Timeout applies to each individual attempt
  - Retry orchestrates multiple attempts with backoff

The Resilience Stack

- Configure all three
  - The circuit breaker checks state
    - If open, fail immediately without making a network call
  - If closed
    - The request goes through with a timeout applied to each individual attempt
  - If the attempt times out or gets a retryable error
    - Retry kicks in
    - Each retry uses exponential backoff with jitter
  - If the retry limit is exhausted
    - The exception propagates back to the circuit breaker, which counts it as a failure

```yaml
resilience4j:
  circuitbreaker:
    instances:
      partnerApi:
        failure-rate-threshold: 50
  retry:
    instances:
      partnerApi:
        max-attempts: 3
        wait-duration: 500ms
        enable-exponential-backoff: true
  timelimiter:
    instances:
      partnerApi:
        timeout-duration: 5s
```

Handling 401/403

401/403

- Common mistake: treating both the same
  - Retrying a 403 will never succeed
    - The token is valid but lacks permission
  - Retrying a 401 without refreshing the token will also never succeed
  - Both require distinct handling logic
- Why you receive a 401 on a previously working call
  - Token has expired (exp claim exceeded)
  - Token has been revoked at the authorization server
  - Authorization server has rotated its signing keys (JWKS rotation)
  - Clock skew between your service and the upstream

Response	Meaning	Cause	Action
`401 Unauthorized`	Not authenticated	Token missing, expired, or invalid signature	Obtain a new token and retry
`403 Forbidden`	Authenticated but not authorized	Token valid but missing required scope or role	Do not retry — escalate or fallback

401/403

- Detecting and handling expired tokens proactively
  - Reactive approach
    - Wait for a 401 then refresh
    - Adds latency, the failed request is wasted
    - Acceptable for low-frequency operations
  - Proactive approach
    - Check expiry before sending the request
    - Inspect the JWT exp claim before attaching the token
    - Refresh if within a configurable threshold
    - For example, 30 seconds before expiry
  - No wasted requests

Response	Meaning	Cause	Action
`401 Unauthorized`	Not authenticated	Token missing, expired, or invalid signature	Obtain a new token and retry
`403 Forbidden`	Authenticated but not authorized	Token valid but missing required scope or role	Do not retry — escalate or fallback

401/403

- 30-second threshold
  - Accounts for clock skew between your service and the upstream.
    - If your token expires at exactly t=3600 but the upstream's clock is 10 seconds ahead
    - the upstream will consider the token expired at t=3590 from your perspective
- In production
  - Spring Security's OAuth2AuthorizedClientManager handles this automatically
  - When using the Spring Security OAuth2 client integration
    - It refreshes tokens before expiry using a configurable clock skew setting

Response	Meaning	Cause	Action
`401 Unauthorized`	Not authenticated	Token missing, expired, or invalid signature	Obtain a new token and retry
`403 Forbidden`	Authenticated but not authorized	Token valid but missing required scope or role	Do not retry — escalate or fallback

401/403

- Handling 401
  - Reactive refresh in the Filter Chain
  - Responding to a 401 in the WebClient filter
  - response.releaseBody() must be called to avoid connection leaks
    - Retry only once to avoid infinite refresh loops
    - Log the refresh event without the token value for observability

```
public ExchangeFilterFunction tokenRefreshFilter(TokenProvider tokenProvider) {
    return (request, next) -> next.exchange(request)
        .flatMap(response -> {
            if (response.statusCode() == HttpStatus.UNAUTHORIZED) {
                // Consume and discard the response body to release the connection
                return response.releaseBody()
                    .then(Mono.defer(() -> {
                        tokenProvider.forceRefresh();
                        ClientRequest retryRequest = ClientRequest.from(request)
                            .header(HttpHeaders.AUTHORIZATION,
                                "Bearer " + tokenProvider.getAccessToken())
                            .build();
                        return next.exchange(retryRequest);
                    }));
            }
            return Mono.just(response);
        });
}
```

401/403

- Logic
  - The filter intercepts every response
  - If the response is 401, it releases the response body
    - Required to return the HTTP connection to the pool
  - It forces a token refresh
  - It rebuilds the original request with the new token
  - It resends the request exactly once

```
public ExchangeFilterFunction tokenRefreshFilter(TokenProvider tokenProvider) {
    return (request, next) -> next.exchange(request)
        .flatMap(response -> {
            if (response.statusCode() == HttpStatus.UNAUTHORIZED) {
                // Consume and discard the response body to release the connection
                return response.releaseBody()
                    .then(Mono.defer(() -> {
                        tokenProvider.forceRefresh();
                        ClientRequest retryRequest = ClientRequest.from(request)
                            .header(HttpHeaders.AUTHORIZATION,
                                "Bearer " + tokenProvider.getAccessToken())
                            .build();
                        return next.exchange(retryRequest);
                    }));
            }
            return Mono.just(response);
        });
}
```

401/403

- 403: Authorization Failures
  - A 403 means the call succeeded but access was denied
  - The token is valid so refreshing will not help
    - The service lacks the required scope or role for this operation
    - This is a configuration or permission issue, not a transient failure
  - Operational response to a 403
    - Alert: this likely indicates a misconfiguration or permission change
    - Do not retry in code: This requires a human or deployment action
    - Log sufficient context to diagnose: endpoint, scopes present, scopes required

```
.retrieve()
.onStatus(
    status -> status == HttpStatus.FORBIDDEN,
    response -> response.bodyToMono(String.class)
        .map(body -> new InsufficientScopeException(
            "Access denied to upstream resource. Body: " + body))
)
.onStatus(
    status -> status == HttpStatus.UNAUTHORIZED,
    response -> response.bodyToMono(String.class)
        .map(body -> new TokenExpiredException("Token rejected by upstream"))
)
```

Configuration

- Production-grade WebClient configuration
  - HTTP engine
    - Sets the physical timeout boundaries
  - OAuth2 filter
    - handles token acquisition and attachment automatically
  - Correlation ID filter
    - Adds tracing context to every outbound call
  - Logging filter
    - Safe observability on every request
  - In a real codebase, each filter would be a @Bean itself for testability
    - The WebClient bean is then the composition of all these parts

```java
@Configuration
public class PartnerApiConfig {

    @Bean
    public WebClient partnerApiClient(
            OAuth2AuthorizedClientManager clientManager,
            @Value("${partner.api.base-url}") String baseUrl) {

        // 1. Configure HTTP engine with timeouts
        HttpClient httpClient = HttpClient.create()
            .option(ChannelOption.CONNECT_TIMEOUT_MILLIS, 3_000)
            .responseTimeout(Duration.ofSeconds(10));

        // 2. Configure OAuth2 token filter
        var oauth2Filter = new ServletOAuth2AuthorizedClientExchangeFilterFunction(clientManager);
        oauth2Filter.setDefaultClientRegistrationId("partner-api");

        // 3. Assemble WebClient with all filters
        return WebClient.builder()
            .baseUrl(baseUrl)
            .clientConnector(new ReactorClientHttpConnector(httpClient))
            .apply(oauth2Filter.oauth2Configuration())
            .filter(correlationIdFilter())
            .filter(loggingFilter())
            .defaultHeader(HttpHeaders.ACCEPT, MediaType.APPLICATION_JSON_VALUE)
            .build();
    }
}
```

Questions?